

UNIVERSIDAD TECNOLÓGICA DEL VALLE DEL MEZQUITAL
PRACTICA DE GIT CORRESPONDIENTE A LA UNIDAD 2 DE LA MATERIA
DESARROLLO WEB INTEGRAL

NOTA: Ir documentando de acuerdo a las evidencias solicitadas. Documenta cada punto de acuerdo a lo que te aparezca en terminal. **Ponderación:** 15% Fecha de entrega: Miercoles 10 de junio – avance y Jueves 11 de junio entrega final a las 12:00hrs en el siguiente drive:

<https://drive.google.com/drive/folders/12PMeq02nVslRwnqI21XZcr2rBrDMYOov?usp=sharing>

Práctica Integral de Git: Control de Versiones y Gestión de Ramas

Objetivo: Aplicar los principales comandos de Git mediante la creación y administración de un proyecto local, utilizando ramas, versionamiento, resolución de conflictos y recuperación de cambios.

Parte 1. Configuración de Git

Verifique la configuración del usuario:
git config --global user.name "Nombre Apellido"
git config --global user.email
"correo@ejemplo.com"
Compruebe la configuración:
git config --list

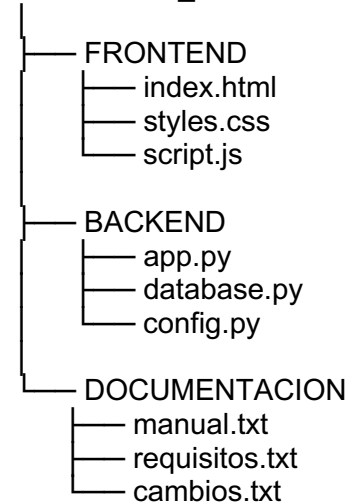
Parte 2. Crear el proyecto e inicializar Git

Crear la carpeta principal:
mkdir PROYECTO_NOMBRE
cd PROYECTO_NOMBRE
Inicializar el repositorio:
git init

Parte 3. Estructura del proyecto

Crear tres carpetas:
mkdir FRONTEND
mkdir BACKEND
mkdir DOCUMENTACION
Dentro de cada carpeta crear tres archivos.
FRONTEND
touch FRONTEND/index.html
touch FRONTEND/styles.css
touch FRONTEND/script.js
BACKEND
touch BACKEND/app.py
touch BACKEND/database.py
touch BACKEND/config.py
DOCUMENTACION
touch DOCUMENTACION/manual.txt
touch DOCUMENTACION/requisitos.txt
touch DOCUMENTACION/cambios.txt

La estructura deberá quedar así:
PROYECTO_NOMBRE



Parte 4. Primer commit

Agregar contenido a:

- FRONTEND/index.html
- BACKEND/app.py
- DOCUMENTACION/manual.txt

Verificar estado:
git status
Agregar archivos:
git add .
Crear el primer commit:
git commit -m "Primer commit del proyecto"
Consultar historial:
git log

Parte 5. Crear alias

Crear alias para visualizar el historial:
git config --global alias.hist "log --oneline --graph --all"
Ejecutar:
git hist

Parte 6. Uso de .gitignore

Crear:
touch .gitignore
Agregar:
*.log
*.tmp
Crear archivos de prueba:
touch errores.log
touch temporal.tmp
Comprobar que Git los ignora:
git status

Parte 8. Creación de ramas

Visualizar ramas:
git branch
Crear una rama:
git branch desarrollo
Cambiar a ella:
git switch desarrollo
o
git checkout desarrollo
Verificar:
git branch

Parte 10. Desplazamiento entre ramas

Regresar a la rama principal:
git switch master
o
git switch main
Comprobar diferencias entre ramas.
Volver a:
git switch desarrollo

Parte 12. Git stash

Modificar:
DOCUMENTACION/manual.txt
Sin realizar commit:
git stash
Comprobar:
git status
Recuperar cambios:
git stash pop

Parte 7. Uso de git diff

Modificar:
• FRONTEND/index.html
Consultar diferencias:
git diff
Guardar cambios:
git add FRONTEND/index.html
git commit -m "Actualización del index"

Parte 9. Trabajar en una rama

Modificar:
• BACKEND/app.py
Guardar cambios:
git add BACKEND/app.py
git commit -m "Cambios en backend desde rama desarrollo"
Consultar historial:
git log --oneline

Parte 11. Etiquetas (Tags)

Crear una etiqueta:
git tag v1.0
Consultar etiquetas:
git tag
Ver información:
git show v1.0

Parte 13. Merge de ramas

Regresar a la rama principal:
git switch main
Integrar cambios:
git merge desarrollo
Consultar historial:
git hist

Parte 14. Generación de conflictos

Crear dos ramas:

```
git branch prueba1
```

```
git branch prueba2
```

Rama prueba1

```
git switch prueba1
```

Modificar:

```
DOCUMENTACION/cambios.txt
```

Guardar:

```
git add .
```

```
git commit -m "Cambio desde prueba1"
```

Rama prueba2

```
git switch prueba2
```

Modificar la misma línea del archivo:

```
DOCUMENTACION/cambios.txt
```

Guardar:

```
git add .
```

```
git commit -m "Cambio desde prueba2"
```

Provocar conflicto

Ir a la rama principal:

```
git switch main
```

Fusionar:

```
git merge prueba1
```

```
git merge prueba2
```

Resolver manualmente el conflicto eliminando:

```
<<<<<<< HEAD
```

```
=====
```

```
>>>>>>> prueba2
```

Guardar y finalizar:

```
git add DOCUMENTACION/cambios.txt
```

```
git commit -m "Conflicto resuelto"
```

Parte 19. Reintegración y eliminación de ramas

Fusionar las ramas pendientes:

```
git merge prueba1
```

```
git merge prueba2
```

Eliminar ramas:

```
git branch -d desarrollo
```

```
git branch -d prueba1
```

```
git branch -d prueba2
```

Verificar:

```
git branch
```

Evidencias solicitadas

1. Captura de la estructura completa del proyecto.

2. Salida de:

```
git status
```

3. Salida de:

```
git log --oneline --graph
```

4. Captura del conflicto generado y su resolución.

Parte 15. Git reset

Modificar:

```
FRONTEND/styles.css
```

Agregar al área de preparación:

```
git add FRONTEND/styles.css
```

Quitar del staging:

```
git reset FRONTEND/styles.css
```

Verificar:

```
git status
```

Parte 16. Git checkout

Recuperar la última versión del archivo:

```
git checkout -- FRONTEND/styles.css
```

Parte 17. git reset --hard

Crear un commit adicional:

```
git add .
```

```
git commit -m "Commit temporal"
```

Consultar historial:

```
git log --oneline
```

Regresar al commit anterior:

```
git reset --hard HEAD~1
```

Parte 18. Recuperar cambios con reflog

Visualizar movimientos:

```
git reflog
```

Recuperar el commit eliminado:

```
git reset --hard ID_COMMIT
```

5. Evidencia del uso de:

- git diff
- git stash
- git tag
- git reflog
- git reset --hard

6. Captura final mostrando únicamente la rama principal activa.